

Multi – level Linked List – Print List(Main List and Child)
–Growth Rate Of Output

Program:

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}
...
case 9:
    cout << "Current List Structure: " << endl;
    printList(head);
    break;
```

Growth Rate of Output

Let $P(n)$ = number of nodes successfully printed to the console.

Each node contributes exactly one output line:

|-- data

and is printed exactly once.

Suppose at some node

- c = number of nodes in its child subtree,
- r = number of nodes reachable through the next pointer.

Then the output produced is

- all output from the child subtree,
- all output from the next subtree,
- one line for the current node.

Although `printList` uses a while loop to traverse the next pointers

(unlike the purely recursive `destroyNodeRecursively`), the mathematical counting of nodes visited and processed remains exactly the same. From the perspective of the starting node, the algorithm eventually prints:

1. All nodes in the child subtree (via the recursive call).
2. All nodes in the remaining next substructure (via the while loop advancing `node = node->next`).
3. The current node itself (via the `cout << "| -- " << node->data` command).

Therefore,

$$P(n) = P(c) + P(r) + 1$$

Meaning of Each Term

1. $P(\text{node} \rightarrow \text{child})$

Represents every output line produced while recursively printing the entire child list.

2. $P(\text{node} \rightarrow \text{next})$

Represents every output line produced while traversing the remaining nodes of the current level.

3. +1

Represents printing the current node itself.

Namely,

```
cout << "|-- " << node->data << endl;
```

This contributes exactly one output line.

$$P(n) = P(c) + P(r) + 1$$

$$\text{or, } P(n) = P(\text{node} \rightarrow \text{child}) + P(\text{node} \rightarrow \text{next}) + 1$$

Structural Equation

Since,

$$n = c + r + 1$$

Substitute into

$$P(n) = P(c) + P(r) + 1$$

By Induction:

$$P(c) = c$$

$$P(r) = r$$

Hence,

$$\begin{aligned} P(n) &= c + r + 1 \\ &= n \end{aligned}$$

Therefore,

$$P(n) = n$$

Growth of Printed Characters

Before printing each node, the function executes

```
for (int i = 0; i < level; i++)
    cout << "    ";
cout << "|-- " << node->data << endl;
```

Suppose a node is at depth d .

Then its line contains

- *4d spaces,*
- *"|--" (4 characters),*
- *the node value.*

Therefore,

characters printed for one node are approximately:

$$4d + O(1)$$

then total output characters are

$$C(n) = \sum_{i=1}^n (4d_i + O(1))$$

Best Case

If there is no child node ,

$A \rightarrow B \rightarrow C \rightarrow D$

then:

$$d_i = 0$$

for every node.

Hence,

$$= 4 \times 0 + \sum_{i=1}^n 1$$

$$= 0 + n$$

$$= \Theta(n)$$

$$\therefore C(n) = \Theta(n)$$

Balanced Multi – level List

$$C(n) = \sum_{i=1}^n (4d_i + O(1))$$

$$= 4 \sum_{i=1}^n d_i + \sum_{i=1}^n O(1)$$

$$\text{Maximum depth} = \log_2(n + 1) = \Theta(\log_2 n)$$

let the depths of the `n` nodes be:

$$d_1, d_2, \dots, d_n$$

The average depth is defined as :

$$\text{Average depth} = \frac{d_1 + d_2 + \dots + d_n}{n} = \frac{\sum_{i=1}^n d_i}{n}$$

$$\therefore \sum_{i=1}^n d_i = n \times \frac{\sum_{i=1}^n d_i}{n} \text{ or, } n \times (\text{Average Depth})$$

Level 0 = $2^0 = 1$ node.

Level 1 = $2^1 = 2$ nodes.

.

.

Level h = 2^h nodes

Total number of nodes = n =

$$1 + 2 + 4 + \dots + 2^h$$

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^n = a \left(\frac{r^{n+1} - 1}{r - 1} \right), \text{ where}$$

S(n) is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

$$1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + \dots + 2^h$$

We get:

$$S(n) = 1 \left(\frac{2^{h+1} - 1}{2 - 1} \right) = 2^{h+1} - 1$$

$$n = 2^{h+1} - 1$$

$$\therefore n + 1 = 2^{h+1}$$

$$\Rightarrow \log_2(n + 1) = \log_2 2^{h+1}$$

$$\Rightarrow \log_2(n + 1) = h + 1$$

$$\Rightarrow \log_2(n + 1) - 1 = h$$

$$\text{or, } h = \log_2(n + 1) - 1$$

$$\therefore \text{Maximum Depth} = h = \log_2(n + 1) - 1$$

Why $\log_2(n + 1) - 1$?

Depth 0 $\rightarrow 2^0$ nodes

Depth 1 $\rightarrow 2^1$ nodes

.

.

Depth $h \rightarrow 2^h$ nodes

As Depth start from 0 nodes ,matching with the series:

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^n = a \left(\frac{r^{n+1} - 1}{r - 1} \right)$$

Hence ,total number of nodes, $n = 2^{h+1} - 1$

$$\therefore \text{Maximum depth} = h = \log_2(n + 1) - 1$$

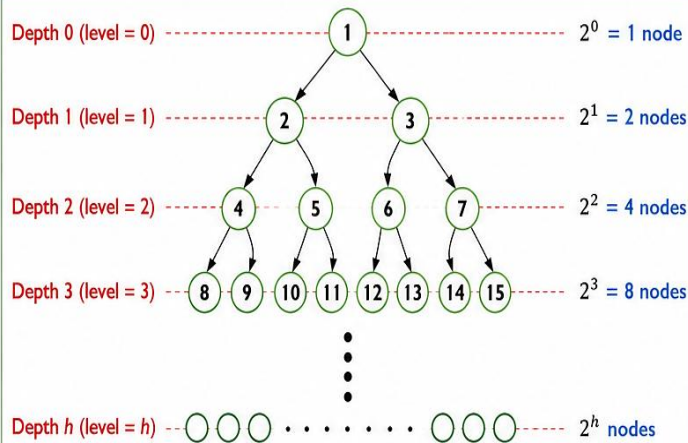
Now,why depth = 0 , as initial call:

printlist(head, level = 0), as level is made with 0 then

depth is 0, then level = 1 , depth = 1 ; level = 2 ,

depth = 2 and it goes on

1. Nodes at each depth (perfect balanced binary tree)



Total number of nodes, $n = 2^0 + 2^1 + 2^2 + \dots + 2^h$

2. Total number of nodes

$$S(n) = 2^0 + 2^1 + 2^2 + \dots + 2^h$$

$$S(n) = a \frac{(r^{n+1} - 1)}{r - 1} \quad [a = 1, r = 2, n = h]$$

$$S(n) = 1 \frac{(2^{h+1} - 1)}{2 - 1}$$

$$S(n) = 2^{h+1} - 1$$

3. Maximum depth h

$$n = 2^{h+1} - 1$$

$$n + 1 = 2^{h+1}$$

$$\log_2(n + 1) = h + 1$$

$$h = \log_2(n + 1) - 1$$

4. Why depth starts from 0?

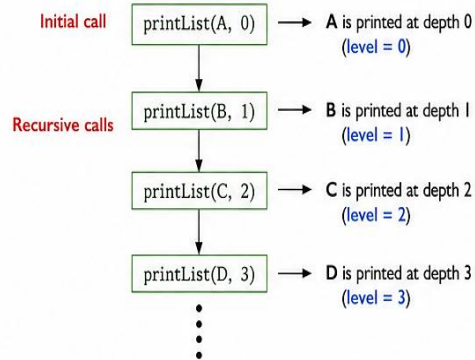
Because the initial call is `printList(head, 0);`

- level = 0 → root at depth 0
- level = 1 → children at depth 1
- level = 2 → grandchildren at depth 2
- ... and so on

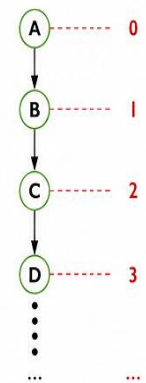
5. How level (depth) increases in the recursive call (Updated Program)

```
void printList(Node* node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "  "; // indentation
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1); // next level
        }
        node = node->next; // next sibling
    }
}
```



Depth (level)



Depth starts from 0 because of `printList(head, 0).`

Nodes at depth $i = 2^i$

Max depth $h = \log_2(n + 1) - 1$

$$4 \sum_{i=1}^n d_i + \sum_{i=1}^n O(1)$$

$$= 4 \sum_{i=1}^n d_i + \Theta(n)$$

$$= 4(n \times (\text{average depth})) + \Theta(n)$$

Each node at depth `i` contributes i.

At depth 0:

- ***1 node contributes***

$$0 \times 1 = 0.$$

At depth 1:

- ***2 nodes contribute***

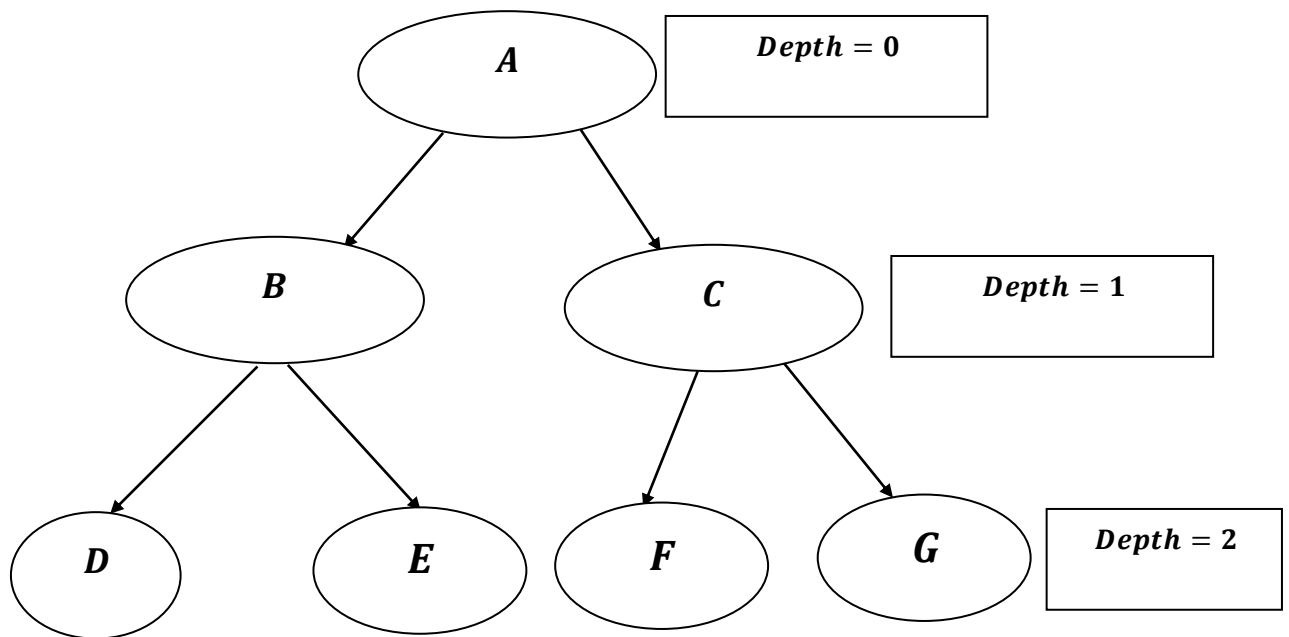
$$1 + 1 = 1 \times 2.$$

At depth 2:

- ***4 nodes contribute***

$$2 + 2 + 2 + 2 = 2 \times 4.$$

What does this mean? Lets check for depth 2:



The nodes at depth 2 are:

- **D** (depth = 2)
- **E** (depth = 2)
- **F** (depth = 2)
- **G** (depth = 2)

Their total contribution to the sum of depths is:

$$2 + 2 + 2 + 2.$$

Therefore,

$$\sum d_i = 0 \times 2^0 + 1 \times 2^1 + 2 \times 2^2 + \dots + h \times 2^h$$

$$\sum d_i = \sum_{i=0}^h i2^i,$$

We have already found out at Recurrence Tree

very 1st document that : $\sum_{i=0}^k i \times 2^i = 2^{k+1}(k-1) + 2$

here, $\sum_{i=0}^h i2^i = 2^{h+1}(h-1) + 2$ or $(h-1)2^{h+1} + 2$.

We know,

$h = \log_2(n+1) - 1$, therefore,

$$= (\log_2(n+1) - 1 - 1)2^{\log_2(n+1)-1+1} + 2.$$

$$= (\log_2(n+1) - 1 - 1)(n+1) + 2.$$

$$= (\log_2(n+1) - 2)(n+1) + 2.$$

$$= \Theta(n \log_2 n)$$

Now, $4(n \times (\text{average depth})) + \Theta(n)$

$$= 4 \left(n \times \frac{\sum d_i}{n} \right) + \Theta(n)$$

$$= 4 \left(n \times \frac{\Theta(n \log_2 n)}{n} \right) + \Theta(n)$$

$$= 4 \left(n \times \Theta(\log_2 n) \right) + \Theta(n)$$

$$= 4 \left(\Theta(n \log_2 n) \right) + \Theta(n)$$

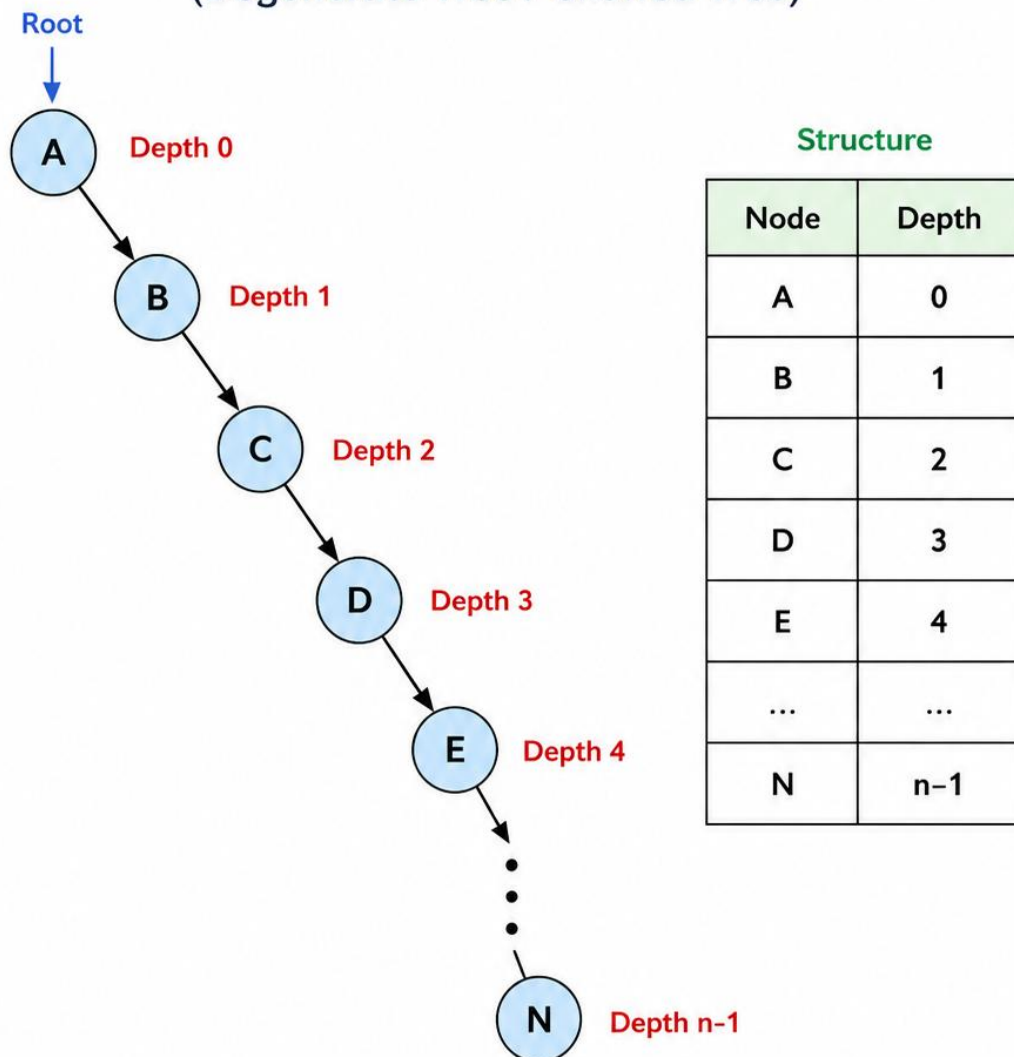
$$= \Theta(n \log_2 n)$$

$$\text{Balanced Multi – level List} = \Theta(n \log_2 n)$$

Worst Case

Suppose every node is a child of the previous node.

Every node is a child of the previous node (Degenerate Tree / Skewed Tree)



Properties

- Each node has exactly one child (except the last node).
- It looks like a single chain.
- Number of nodes = n
- Height of the tree = $n - 1$
- This is the worst case for height (or depth).

Example for $n = 6$

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Depths: 0, 1, 2, 3, 4, 5

Height = $5 = n - 1$

$$4 \sum_{i=1}^n d_i + \sum_{i=1}^n O(1)$$

$$= 4 \sum_{i=1}^n d_i + \Theta(n)$$

$$\text{or, } \sum d_i = 0 + 1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2}$$

$$= 4 \left(\frac{n(n - 1)}{2} \right) + \Theta(n)$$

$$= \Theta(n^2)$$

Hence,

$$C(n) = \Theta(n^2)$$

Final Output Growth

There are two different notions of output growth:

(1) *Number of output lines*

Let

$P(n)$ = *number of printed nodes.*

Then:

$$P(n) = P(c) + P(r) + 1 = n = \Theta(n).$$

(2) *Number of output characters (including indentation)*

Because indentation depends on the depth,

$$C(n) = 4 \sum_{\text{all nodes}} (\text{depth}) + \Theta(n)$$

Therefore:

- *Best case:* $\Theta(n)$
- *Balanced multi – level list:* $\Theta(n \log n)$
- *Worst case (completely nested child chain):* $\Theta(n^2)$